ACADEMIC MAJOR PROJECT TECHNICAL DOCUMENTATION

CLABTOPro

Deterministic C Online Judge & Proctoring Platform

An enterprise-grade, MERN-stack C programming laboratory solution featuring isolated containerized execution, multi-worker concurrency queuing, deterministic output normalization, evidence-based AI debugging guidance, and full-window anti-cheat proctoring.

PROJECT HIGHLIGHTS

- Deterministic GCC Online Judge
- Isolated Piston Docker Sandbox
- Local Host GCC Fallback Engine
- Google Gemini AI Debug Assistant
- Active Tab & App Switching Proctoring

TARGET DEPLOYMENT SPECS

- Target Users: ~120 Students
- Stack: React, Vite, Node, Express, Mongo
- Cloud Hosts: Render (API) + Vercel (UI)
- Sandbox: Linux Containers / GCC 13
- Status: Production Verified

PROJECT & INSTITUTION DETAILS

Author / Developer:	Raju (College Major Project Team)
Course & Department:	Computer Science & Engineering (B.Tech)
Live Backend:	**https://clabtopro-backend-ehns.onrender.com**
Documentation Date:	October 2026

CHAPTER 1

Executive Summary & Core Objectives

1.1 Background & Problem Statement

In many contemporary educational coding platforms, code evaluation is simulated using Large Language Models (LLMs) that predict whether source code satisfies test cases. While LLMs are powerful natural language processors, relying on AI models to judge student code leads to significant issues: hallucinations, acceptance of non-compiling syntax, false rejections of valid algorithmic variants, and non-deterministic scoring.

CLABTOPro completely replaces simulated AI judging with an authoritative, deterministic Online Judge architecture. The system physically compiles and runs student C code inside an isolated sandbox, enforces resource limits (CPU, memory, wall time), normalizes program standard output, and evaluates submissions deterministically. The LLM is strictly decoupled as an educational mentor that provides failure explanations and debugging guidance without overriding official verdicts.

1.2 Key System Objectives

- **Deterministic Evaluation:** Compile with real GCC 13 and execute student code against exact problem test cases.
- **Isolated Execution Sandbox:** Execute untrusted student code inside isolated Piston/Docker containers with zero host contamination.
- **Dual-Engine Architecture:** Primary execution through Piston Docker with an automatic, zero-downtime Local GCC Sandbox fallback.
- **Controlled Concurrency:** Handle 120 concurrent college students with configurable queue workers (EXECUTION_CONCURRENCY=4).
- **Evidence-Based AI Explainer:** Call Google Gemini only when code fails, feeding actual compiler diagnostic evidence or output diffs.
- **Comprehensive Proctoring:** Monitor and restrict browser tab switching and background application switches (Alt+Tab) across all student views.

> **ARCHITECTURAL PRINCIPLE: ZERO HOST COMPROMISE**
> Student code is inherently untrusted. CLABTOPro enforces execution timeouts (3-5s), output caps (64KB), memory bounds, and strictly avoids unmonitored child process execution on production hosts.

CHAPTER 2

End-to-End System Architecture

2.1 High-Level Request Pipeline

The CLABTOPro architecture connects a responsive React SPA to a robust Node.js/Express API. All judging tasks are routed through a concurrency-controlled queue to prevent server thrashing during laboratory peaks.

```
[Student React Editor]
         %  (POST /api/submissions)
         %¼
[Express Backend API] % % %º [JWT Authentication & Validation]
         %
         %¼
[Submission Queue Worker] % % %º Concurrency Control (Max 4 active workers)
         %
         % % % %º [Primary Engine]: Isolated Piston Docker Sandbox (Port 2000)
         %         % %  (If offline/unreachable)
         % % % %º [Fallback Engine]: Sandboxed Local GCC 13 Runner (/tmp/clabto-*)
         %
         %¼
[Deterministic Verdict Engine]
         % % %  Output Normalization (CRLF -> LF, whitespace trimming)
         % % %  Compare Actual vs Expected Outputs
         %¼
[MongoDB Atlas] %Ä% %  [Save Official Submission Record & Progress Streak]
         %
         % % %  (If Failed: Wrong Answer / Compile / Runtime / Timeout)
         %¼
[Google Gemini AI Assistant] % % %º Multi-Model Failover -> Structured Hints
         %
         %¼
[Student UI Instant Feedback] (Verdict Banner, Progress Bar, Test Breakdown)
```

2.2 Component Directory Structure

The codebase is modularized cleanly across frontend, backend services, models, and deployment configurations:

- **backend/services/judgeService.js:** Coordinates engine health, test-case iteration, and deterministic verdict comparison.
- **backend/services/pistonService.js:** Handles HTTP REST communication with the Piston Docker container.
- **backend/services/localGccService.js:** Sandboxed local execution fallback with EPIPE stream guards and temporary folders.
- **backend/services/submissionQueue.js:** Controls execution concurrency with fair queueing and task scheduling.
- **backend/services/aiEvaluationService.js:** Evidence-based failure explanations with multi-model failover.
- **frontend/src/context/RestrictionContext.jsx:** Monitors visibilitychange and window blur to prevent cheating.
- **frontend/src/components/common/RestrictionModal.jsx:** Displays immediate violation warnings on tab/app switch.

CHAPTER 3

Deterministic Judging & Output Normalization

3.1 Verdict Decision Flow

Submissions are evaluated through a strict multi-tier verification process. Each stage can assign a definitive verdict:

- **1. Compilation Stage:** Student code is compiled with 'gcc -O2 -Wall -std=c11 -lm'. If exitCode != 0, verdict is COMPILE_ERROR with full compiler diagnostics.
- **2. Execution Stage:** The compiled binary is executed with stdin inputs. If runtime exceeds EXECUTION_TIMEOUT_MS (3000ms), SIGKILL is dispatched and verdict is TIME_LIMIT.
- **3. Runtime Safety:** If exitCode != 0 or fatal signals occur (SIGSEGV, SIGFPE, SIGABRT), verdict is RUNTIME_ERROR.
- **4. Output Comparison:** If execution succeeds, actual stdout is compared against expected output. All passed = ACCEPTED; any failure = WRONG_ANSWER.

3.2 Strict Output Normalization Logic

To eliminate false negatives caused by operating system variations (e.g. Windows CRLF \r\n vs Linux LF \n) or trailing blank lines, the judge normalizes all strings before comparison:

```
export const normalizeOutput = (output) => {
  if (output === null || output === undefined) return "";
  return String(output)
    .replace(/\r\n/g, "\n")
    .replace(/\r/g, "\n")
    .split("\n")
    .map((line) => line.trimEnd())
    .join("\n")
    .trimEnd();
};
```

3.3 Hidden Test Case Security

Problems support both public and hidden test cases. Public cases display actual inputs and outputs to help students understand their mistakes. Hidden test cases securely omit secret inputs and expected values from API payloads, reporting only whether the case passed or failed.

CHAPTER 4

Evidence-Based AI Learning Assistant

4.1 Role of Artificial Intelligence in CLABTOPro

Unlike legacy platforms where AI was mistakenly used as the judge, CLABTOPro uses Google Gemini strictly as an Educational Teaching Assistant. The AI is only invoked when a submission fails, and it is strictly constrained by prompt engineering never to modify the official verdict.

4.2 Multi-Model Failover Architecture

Public AI endpoints frequently encounter high-demand spikes (HTTP 503) or rate limits (HTTP 429). To guarantee 100% platform availability, CLABTOPro implements a multi-model failover chain:

- **Primary Model:** gemini-flash-lite-latest (fast response, high throughput)
- **Fallback Tier 1:** gemini-3.5-flash-lite (production release)
- **Fallback Tier 2:** gemini-3.1-flash-lite (high availability)
- **Fallback Tier 3:** gemini-3.6-flash (multimodal backup)
- **Rule-Based Fallback:** If all cloud APIs are unreachable, generate deterministic diagnostic hints directly from GCC output and exit codes.

RELIABILITY GUARANTEE

Even during complete internet outages or external API blackouts, student submissions will evaluate cleanly, compute streaks, and receive structured diagnostic feedback.

CHAPTER 5

Tab & Application Switch Proctoring

5.1 Threat Model in Academic Computer Labs

During programming examinations and laboratory sessions, students frequently attempt to cheat by switching to AI tools in another browser tab, opening background IDEs, or switching applications via Alt+Tab. CLABTOPro integrates active client-side proctoring.

5.2 Dual-Event Detection Mechanism

- **Tab Switching:** Monitored via the document 'visibilitychange' API. Fires when document.hidden becomes true.
- **App Switching:** Monitored via the window 'blur' API. Fires when focus shifts to another desktop application, recent app switcher, or dock.
- **Debounce Guard:** Rapid-fire events within 800ms are coalesced into a single incident to prevent inflating counts.
- **Full-Screen Warning Modal:** Locks student interaction with a backdrop blur and displays violation counter, event type, and timestamp.
- **Permanent Audit Trail:** Live violation count is submitted to the database with code submissions for instructor review.

```
// In StudentLayout.jsx and ProblemDetails.jsx
const { switchCount } = useRestriction();

// Passed alongside student solution
await submitCode({
  problemId,
  code,
  language: "c",
  tabSwitches: switchCount
});
```

CHAPTER 6

Database Schemas & Data Models

6.1 Core Mongoose Schemas

CLABTOPro utilizes five relational-style collections in MongoDB Atlas:

- **User:** Stores student and admin credentials, hashed passwords, roles, currentStreak, longestStreak, and lastSolvedDate.
- **Problem:** Defines problem title, description, constraints, sampleInput, sampleOutput, and testCases (with isHidden flag).
- **Submission:** Records studentId, problemId, code, language, status, testCasesPassed, totalTestCases, executionTime, and tabSwitches.
- **Progress:** Aggregates overall student performance: totalSolved, totalAccepted, totalAttempted, and daily streaks.
- **Topic:** Categorizes curriculum topics: Basics, Loops, Arrays, Strings, Pointers, Functions.

6.2 Data Sanitization Pattern

To protect exam integrity, the backend applies 'sanitizeSubmissionForStudent' before returning submission documents over the network. Raw test case results for hidden tests are stripped of their inputs and expected outputs, while preserving testCaseNumber, status, and executionTime.

CHAPTER 7

API Reference & Production Deployment

7.1 Key REST Endpoints

- **POST /api/auth/login:** Authenticates student/admin and returns JWT token.
- **GET /api/problems:** Retrieves problem catalog filtered by student progress.
- **GET /api/problems/:id:** Fetches problem details and previous student submission.
- **POST /api/submissions:** Enqueues C code for isolated execution and deterministic verdict.
- **GET /api/execution/health:** Live health status of execution engine and queue metrics.
- **GET /api/admin/students:** Instructor dashboard tracking student submissions and tab violations.

7.2 Production Cloud Deployment (Live Verified)

CLABTOPro is fully deployed and production-verified across modern cloud infrastructure:

- **Frontend (Vercel):** React 18 SPA built with Vite. Optimized static distribution on global edge CDN.
- **Backend (Render):** Docker container running Node.js 20 with Debian GCC 12/13 tools installed.
- **Database (MongoDB Atlas):** Clustered cloud database with TLS 1.3 encryption and automated backups.
- **Live Backend URL:** https://clabtopro-backend-ehns.onrender.com

DEPLOYMENT VERIFICATION STATUS

Engine Status: HEALTHY | Compiler: gcc (Debian 12.2.0) | Concurrency: 4 workers | Memory Isolation: Active

CHAPTER 8

Conclusion & Future Enhancements

8.1 Summary of Contributions

CLABTOPro provides college computing laboratories with an industrial-strength coding and judging ecosystem. By shifting from simulated AI predictions to actual GCC sandbox execution, students receive reliable, deterministic feedback on their code while instructors gain complete visibility into student progress and academic integrity.

8.2 Future Roadmap

- **Multi-Language Support:** Extend sandbox runtimes to C++, Java, and Python.
- **Distributed Queue:** Replace in-memory queue with Redis and BullMQ for multi-server scalability beyond 500 concurrent students.
- **Automated Code Plagiarism:** Integrate AST-based code similarity algorithms (e.g. MOSS) for cheating detection across student submissions.
- **Real-Time Code Collaboration:** WebSocket-enabled collaborative debugging between instructors and struggling students.